

Invariantes de ciclo

Correctitud de algoritmos iterativos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Escribir la especificación de un problema: entrada con sus precondiciones y salida con sus poscondiciones (OA6).
- Explicar cuándo un algoritmo es correcto y qué técnica de demostración corresponde a cada familia de algoritmos (OA6).
- Proponer los invariantes I_0 e I_1 de un ciclo a partir de su traza de estados (OA1, OA6).
- Demostrar invariantes por inicialización y estabilidad, y usar la terminación para concluir la correctitud del algoritmo (OA6).

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

¿Qué promete un algoritmo?

Antes de demostrar, hay que prometer

Para poder afirmar que un algoritmo está bien, primero hay que decir con precisión qué problema resuelve. Esa promesa es la **especificación del problema**, y tiene dos elementos:

- **Entrada:** especificación formal de los datos de entrada y las condiciones que deben cumplir (las *precondiciones*).
- **Salida:** especificación formal del resultado esperado (las *poscondiciones*).

Instancias y correctitud

Una **instancia** del problema es una asignación de valores para los datos de entrada. Un algoritmo es **correcto** si calcula el resultado correcto para *todas* las instancias del problema.

Dos técnicas para dos familias

Se desea demostrar que los algoritmos que se diseñan son correctos

- **Inducción estructural** — para los algoritmos recursivos (dividir y conquistar). La veremos con esa técnica.
- **Análisis de invariantes de ciclo** — para los algoritmos iterativos. Es el tema de hoy.

La especificación más corta del curso: el factorial

- **Entrada:** $N \in \{\mathbb{N}, \perp\}$
- **Salida:** $N!$.

Ahora usted: especifique la búsqueda

Ejercicio rápido

Dado un arreglo de números y un valor, se quiere decidir si el valor aparece en el arreglo. Escriba la especificación: ¿qué entra, con qué condiciones, y qué sale?

Ahora usted: especifique la búsqueda. Respuesta

Búsqueda en un arreglo arbitrario

- **Entrada:** un arreglo $A[0..N)$ de números, con $N \in \mathbb{N}$, y un número v .
- **Salida:** $\exists p \in [0..N). A[p] = v$.

Dos detalles de notación

El rango $[0..N)$ es *semiabierto*: incluye 0 y excluye N , igual que los índices válidos del arreglo. Y la salida es una fórmula lógica: el algoritmo debe devolver verdadero exactamente cuando exista una posición p con $A[p] = v$.

Variantes del mismo problema

Búsqueda que entrega la posición

- **Entrada:** un arreglo $A[0..N)$ de números y un número v .
- **Salida:** p si $\exists p \in [0..N). A[p] = v$; -1 si no existe tal p .

Búsqueda en un arreglo ordenado

- **Entrada:** un arreglo $A[0..N)$ de números **ordenado ascendentemente** y un número v .
- **Salida:** $\exists p \in [0..N). A[p] = v$.

Variantes del mismo problema: para pensar

Para pensar

La tercera pide lo mismo que la primera, pero la entrada gana una precondition. Una precondition más fuerte abre la puerta a algoritmos más rápidos: a eso volveremos con la búsqueda binaria.

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

Un ciclo bajo la lupa

El problema

Entrada: un arreglo $A[0..N)$ de números, $N \geq 0$. **Salida:** $\sum_{j=0}^{N-1} A[j]$.

```
def sumarArreglo(A):  
    i = 0  
    ans = 0  
    while i < len(A):  
        ans = ans + A[i]  
        i = i + 1  
    return ans
```

Un ciclo bajo la lupa

Antes de ejecutar

Las **variables** del ciclo son i , el índice, y ans , el acumulador del resultado. Un **estado** es la tupla (i, ans) con los valores de las variables en un chequeo de la condición. ¿Qué devuelve `sumarArreglo([9, 4, -5, 1, 8, 3])`? Siga a mano el estado (i, ans) cada vez que el `while` evalúa su condición.

$ans = 20$

```
def sumarArreglo(A):  
    i = 0  
    ans = 0  
    while i < len(A):  
        ans = ans + A[i]  
        i = i + 1  
    return ans
```

1. Que cambia en el algoritmo
2. Mirar su evolucion

(i, ans) Estado inicial
 $(0, 0) \rightarrow (1, 9) \rightarrow (2, 13) \rightarrow$
 $(3, 8) \rightarrow (4, 9) \rightarrow (5, 17) \rightarrow$
 $(6, 20)$ Estado final

Un ciclo bajo la lupa. Respuesta

Chequeo	(i, ans)	ans acumula
1	(0, 0)	nada todavía
2	(1, 9)	$A[0]$
3	(2, 13)	$A[0] + A[1]$
4	(3, 8)	$A[0] + A[1] + A[2]$
5	(4, 9)	$A[0] + \dots + A[3]$
6	(5, 17)	$A[0] + \dots + A[4]$
7	(6, 20)	$A[0] + \dots + A[5]$

ans

$j \leq 4$

$$\text{ans} = \sum_{j=0}^{4-1} A[j]$$

$$\sum_{j=0}^{4-1} A[j] + A[j]$$

$$(A[0] + A[1] + A[2] + A[3]) + A[4]$$

$i = 4+1$ $i = 5$

Un ciclo bajo la lupa. Respuesta

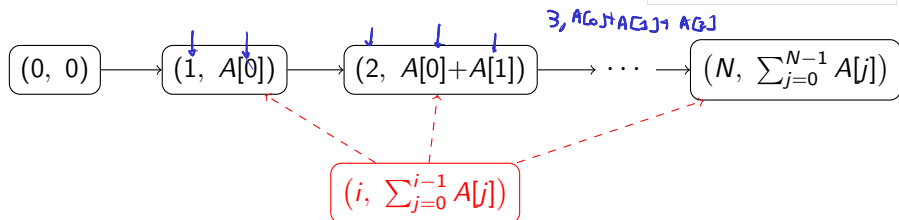
El patrón

Devuelve 20. Pero lo importante está en la tercera columna: en cada chequeo, `ans` lleva la suma de los primeros i elementos —incluso baja cuando entra el -5 , y la propiedad se sigue cumpliendo— y además i nunca se sale del rango $[0, N]$. Dos propiedades que *todas* las filas cumplen.

La cadena de estados y su generalización

```
def sumarArreglo(A):  
    {  
        i = 0  
        ans = 0  
        while i < len(A):  
            ans = ans + A[i]  
            i = i + 1  
        return ans  
    }
```

$(i, \text{ans}) \quad A[0, n) \in \mathbb{Z}$



Del caso particular al caso general

La tabla anterior es el **caso particular** $A = [9, 4, -5, 1, 8, 3]$; la cadena de arriba es el **caso general**: parte del **estado inicial** $(0, 0)$ y llega al **estado final** $(N, \sum_{j=0}^{N-1} A[j])$, la poscondición.

La cadena de estados y su generalización

Invariante de ciclo

Un **invariante** corresponde a una propiedad sobre las variables que debe ser cierta a lo largo de la ejecución de un ciclo. La forma general en rojo es eso: la propiedad que cada estado particular de la cadena satisface.

Los dos invariantes del ciclo

Siempre en pareja

$$I_0 : 0 \leq i \leq N \qquad I_1 : \text{ans} = \sum_{j=0}^{i-1} A[j]$$

- I_0 **acota el índice**: dice por dónde puede ir i .
- I_1 habla del **acumulador**: dice qué lleva calculado ans en función de i .

Ninguno sobra

Sin I_1 no se sabe qué calcula el ciclo; sin I_0 no se sabe dónde termina i , y al final es la combinación de ambos la que entrega la poscondición.

El método

Para mostrar que un invariante se cumple se debe demostrar que

- 1 La fórmula lógica es verdadera antes de la primera iteración.
(Inicialización)
- 2 Si la fórmula lógica es cierta antes de cualquier iteración, debe seguir siendo cierta después de la iteración. (Estabilidad)
- 3 El ciclo debe terminar y los invariantes deben suministrar información importante acerca del objetivo del ciclo y del algoritmo.
(Terminación)

Si lo busca en CLRS

CLRS (pp. 18–20) llama mantenimiento a la estabilidad; la terminación conserva su nombre. Es el mismo método con otro rótulo.

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo**
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

La suma: Teorema 1 e inicialización

$$I_0: 0 \leq i \leq N \quad I_1: \text{ans} = \sum_{j=0}^{i-1} A[j] \quad (0, 0) \\ (i, \text{ans})$$

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

```
def sumarArreglo(A):  
    i = 0  
    ans = 0  
    while i < len(A):  
        ans = ans + A[i]  
        i = i + 1  
    return ans
```

Inicialización

De acuerdo a las líneas 1–2, inicialmente $i = 0$ y $\text{ans} = 0$. Para I_0 : $0 \leq 0 \leq N$. ✓ Para I_1 : la suma $\sum_{j=0}^{-1} A[j]$ recorre el rango vacío $[0, 0)$ y por convención vale $0 = \text{ans}$. ✓ Por lo tanto, los invariantes I_0 e I_1 se cumplen en la inicialización.

La suma: la estabilidad

```

0 def sumarArreglo(A):
1     i = 0
2     ans = 0
3     while i < len(A):
4         {ans = ans + A[i]
5           i = i + 1}
6     return ans

```

$$I_0: 0 \leq i \leq N \quad I_1: \text{ans} = \sum_{j=0}^{i-1} A[j] \quad \begin{pmatrix} 0 & 0 \\ i & \text{ans} \end{pmatrix}$$

$i=j \quad i \neq N \quad i \neq 0$

Estabilidad

Se considera una iteración arbitraria $i = j$, diferente a la última (con $j < N$: de lo contrario el cuerpo no se habría ejecutado), y se asume que antes de ella valen $0 \leq j \leq N$ y $\text{ans} = \sum_{j'=0}^{j-1} A[j']$. Al ejecutar las líneas 4-5:

$i=j$ Lo que llevo hasta el momento

$$\text{ans} = \sum_{j'=0}^{j-1} A[j'] + A[j] = \sum_{j'=0}^j A[j'], \quad \text{circled } i = j + 1, \quad \left(j, \sum_{j'=0}^{j-1} A[j'] \right)$$

que es I_1 evaluado en $j+1$. Para I_0 : de $j < N$ sale $0 \leq j+1 \leq N$. Por lo tanto, los invariantes son estables. ✓

$i=j+1$

$$\sum_{i=0}^{j-1} A[i] \rightarrow \sum_{i=0}^j A[i]$$

La suma: la terminación

```
def sumarArreglo(A):  
    i = 0  
    ans = 0  
    while i < len(A):  
        ans = ans + A[i]  
        i = i + 1  
    return ans
```

Terminación

Se puede garantizar que el ciclo va a terminar ya que i se incrementa de 1 en 1 e I_0 lo acota por N . Finalmente, en la iteración en la que termina el ciclo la condición $i < N$ es falsa, o sea $i \geq N$; junto con I_0 esto da exactamente $i = N$. Sustituyendo ese valor en I_1 :

$$\text{ans} = \sum_{j=0}^{N-1} A[j]$$

lo que coincide con la poscondición. Por lo tanto, los invariantes son correctos.



La suma: el teorema del algoritmo

Teorema 2

La invocación `sumarArreglo(A)` para cualquier arreglo de números $A[0..N]$ con $N \geq 0$ produce como resultado la suma de los elementos en $A[0..N]$.

Demostración: es trivial a partir de la correctitud de los invariantes I_0 e I_1 (Teorema 1): la terminación deja $i = N$, y sustituir ese valor en I_1 da la poscondición, que es lo que retorna la línea 6. ■

La suma: el caso límite

¿Y si el arreglo viene vacío?

Con $N = 0$ la condición de la línea 3 falla desde el primer chequeo: el ciclo no corre y la función devuelve 0. No hay nada que arreglar. La especificación admite $N \geq 0$ y la suma sobre el rango vacío vale 0, así que la poscondición se cumple; de hecho la inicialización ya establece los invariantes antes de la primera evaluación, y la terminación los lee ahí mismo con $i = 0 = N$.

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial**
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

El factorial, ahora con ciclo

```
def fact(N):
```

```
    ans = 1
```

```
    i = 1
```

```
    while i <= N:
```

```
        ans = ans * i
```

```
        i = i + 1
```

```
    return ans
```

$ans = (i-1)!$

$ans = 0!$

$6! = 1 \times 2 \times 3 \times 4 \times$

$N! = 1 \times 2 \times 3 \times 4 \times \dots$

$i = i + 1$ (f)

La cadena de estados (i, ans) para $N = 5$

$1 \leq i \leq N+1$

$ans = (i-1)!$

$0! = 1$

$(1, \underline{1}) \rightarrow (2, \underline{1}) \rightarrow (3, \underline{2}) \rightarrow (4, \underline{6}) \rightarrow (5, \underline{24}) \rightarrow (6, \underline{120})$

$(1, 0!)$

$(2, 1!)$

$(3, 2!)$

$(4, 3!)$

$(5, 4!)$

$(6, 5!)$

Ahora usted

Proponga los dos invariantes: ¿entre qué valores se mueve i ? ¿Qué lleva ans en función de i ?

El factorial, ahora con ciclo. Respuesta

Los invariantes

$$I_0 : 1 \leq i \leq N + 1$$

$$I_1 : \text{ans} = (i - 1)!$$

¿Por qué $N + 1$ y por qué $(i - 1)!$?

La condición es $i \leq N$, así que el último chequeo ocurre con $i = N + 1$: la cota superior de I_0 debe dejarlo entrar. Y en cada chequeo `ans` lleva multiplicados los factores $1, 2, \dots, i - 1$ (el factor i aún no entra): por eso $(i - 1)!$, como se ve en la cadena: en $(3, 2)$, $\text{ans} = 2 = 2!$.

El factorial frente a sumarArreglo

Compare con sumarArreglo

Allá la condición era $i < N$ y el ciclo termina con $i = N$; aquí es $i \leq N$ y termina con $i = N + 1$. La cota superior de I_0 se lee directo de la condición del `while`.

Teorema 1: el enunciado

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Teorema 1: la inicialización

```
def fact(N):  
    ans = 1  
    i = 1  
    while i <= N:  
        ans = ans * i  
        i = i + 1  
    return ans
```

Inicialización

De acuerdo a las líneas 1–2, inicialmente $\text{ans} = 1$ e $i = 1$. Para el invariante I_0 se tiene que

$$1 \leq i \leq N + 1 \longrightarrow 1 \leq 1 \leq N + 1. \checkmark$$

Para el invariante I_1 se tiene que

$$\text{ans} = (i - 1)! \longrightarrow 1 = (1 - 1)! = 0! = 1. \checkmark$$

Por lo tanto, los invariantes I_0 e I_1 se cumplen en la inicialización.

Estabilidad: lo que se asume

```
def fact(N):  
    ans = 1  
    i = 1  
    while i <= N:  
        ans = ans * i  
        i = i + 1  
    return ans
```

Estabilidad: lo que se asume y lo que se quiere

Se considera una iteración arbitraria $i = j$ (con $j \leq N$: de lo contrario no se habrían ejecutado las líneas del ciclo) y se asume que antes de ejecutarla se cumplen los invariantes:

$j \neq 0 \wedge j \neq N+1$

$$1 \leq j \leq N + 1$$

$$\text{ans} = (j - 1)!$$

El objetivo es mostrar que después de esta iteración — o sea, antes de la iteración $i = j + 1$ — se cumplen:

I_o $1 \leq j + 1 \leq N + 1$

$$\text{ans} = j!$$

Estabilidad: el cálculo

$$(j-1)! \times j = j! \quad \checkmark$$
$$(j-2)! \cdot j = j+1 = j! \quad \checkmark$$

```
def fact(N):  
    ans = 1  
    i = 1  
    while i <= N:  
        ans = ans * i  
        i = i + 1  
    return ans
```

Estabilidad: el cálculo

Al ejecutar las líneas 4–5 con $i = j$ y $\text{ans} = (j-1)!$:

$$\text{ans} = \text{ans} \cdot i = (j-1)! \cdot j = j! \quad i = j + 1.$$

El invariante I_0 es verdadero ya que $j \leq N$ implica $1 \leq j+1 \leq N+1$, y el valor $\text{ans} = j!$ satisface I_1 . Por lo tanto, los invariantes son estables. $\checkmark$

La terminación

```
0 def fact(N):  
1     ans = 1  
2     i = 1  
3     while i <= N:  
4         ans = ans * i  
5         i = i + 1  
6     return ans
```

Terminación

Se puede garantizar que el ciclo va a terminar ya que el valor de i se incrementa de 1 en 1 hasta superar a N . Finalmente, en la iteración en la que termina el ciclo se tiene $i = N + 1$; luego, por la estabilidad del invariante I_1 :

$$\text{ans} = (i - 1)! = (N + 1 - 1)! = N!$$

y en consecuencia, cuando se llega a la línea 6, $\text{ans} = N!$. Este valor coincide con la poscondición. Por lo tanto, los invariantes son correctos. ■

El teorema del algoritmo

Teorema 2

La invocación $\text{fact}(N)$ para cualquier $N \geq 0$ produce como resultado $N!$.

Demostración: es trivial a partir de la correctitud de los invariantes I_0 e I_1 (Teorema 1): la terminación deja $i = N + 1$, y sustituir ese valor en I_1 da $\text{ans} = (N + 1 - 1)! = N!$, que es justo lo que retorna la línea 6. ■

¿Y `fact(-1)`?

Fuera de la precondition no hay promesa

Con $N = -1$ la condición $1 \leq N$ falla desde el primer chequeo: el ciclo no corre y la función devuelve 1. ¿Está mal el algoritmo? No: la especificación pide $N \in \mathbb{N}$, y -1 no es una instancia del problema. La correctitud se promete para las entradas que cumplen las precondiciones; para las demás, el algoritmo no debe nada.

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo**
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

El algoritmo de búsqueda

El problema (ya especificado)

Entrada: un arreglo $A[0..N)$ de números, $N \in \mathbb{N}$, y un número v .

Salida: $\exists p \in [0..N). A[p] = v$.

```
1 bool solve(vector<int>& A, int v) {  
2     bool ans = false;  
3     int i = 0;  
4     while (i < A.size()) {  
5         if (A[i] == v)  
6             ans = true;  
7         ++i;  
8     }  
9     return ans;  
}
```

La traza de la búsqueda

$A = [8, 1, 4, 2, 5, 6, 7, 10]$, $N = 8$, $v = 5$

Chequeo	(i, ans)	Revisado hasta ahora
1	$(0, \text{false})$	$[]$
2	$(1, \text{false})$	$[8]$
3	$(2, \text{false})$	$[8, 1]$
4	$(3, \text{false})$	$[8, 1, 4]$
5	$(4, \text{false})$	$[8, 1, 4, 2]$
6	$(5, \text{true})$	$[8, 1, 4, 2, 5]$
$\vdots$	$\vdots$	$\vdots$
9	$(8, \text{true})$	$[8, 1, 4, 2, 5, 6, 7, 10]$

La traza de la búsqueda: ahora usted

Ahora usted

ans no acumula una suma: acumula una *respuesta*. ¿Qué responde exactamente en cada chequeo? Proponga l_0 e l_1 .

La traza de la búsqueda. Respuesta

Los invariantes

$$I_0 : 0 \leq i \leq N$$

$$I_1 : \text{ans} = \exists p \in [0..i). A[p] = v$$

En palabras

ans responde la pregunta del problema, pero *solo sobre el prefijo ya revisado* $A[0..i)$. Cuando i llegue a N , el prefijo será todo el arreglo y ans responderá la pregunta completa: así es como I_1 va a entregar la poscondición.

La demostración de la búsqueda

$I_0: 0 \leq i \leq N$

$I_1: \text{ans} = \exists p \in [0..i). A[p] = v$

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

De acuerdo a las líneas 1-2, inicialmente $\text{ans} = \text{false}$ e $i = 0$. Para I_0 : $0 \leq 0 \leq N$. ✓ Para I_1 :

$$\text{false} = \exists p \in [0.,0). A[p] = v$$

que se cumple dado que $[0.,0)$ es un rango vacío y en consecuencia no puede existir el valor p que haga verdadero el existencial. ✓

```
bool solve(vector<int>& A, int v) {  
    bool ans = false;  
    int i = 0;  
    while (i < A.size()) {  
        if (A[i] == v)  
            ans = true;  
        ++i;  
    }  
    return ans;  
}
```

La demostración de la búsqueda: esta

$$I_0: 0 \leq i \leq N$$

$$I_1: \text{ans} = \exists p \in [0..i). A[p] = v$$

```
bool solve(vector<int>& A, int v) {  
    bool ans = false;  
    int i = 0;  
    while (i < A.size()) {  
        if (A[i] == v)  
            ans = true;  
        ++i;  
    }  
    return ans;  
}
```

Estabilidad

Se considera una iteración arbitraria $i = j$ (con $j < N$) y se asumen $0 \leq j \leq N$ y $\text{ans} = \exists p \in [0..j). A[p] = v$. El objetivo es mostrar que después de la iteración $\text{ans} = \exists p \in [0..j+1). A[p] = v$ y $0 \leq j+1 \leq N$. Al ejecutar las líneas 4–6 hay dos posibilidades:

- $A[j] = v$: ans pasa a valer true, y esto es correcto porque la posición j del arreglo es igual a v : el existencial sobre $[0..j+1)$ es verdadero. ✓
- $A[j] \neq v$: ans conserva el valor que traía, y esto es correcto porque agregar una posición que no es v no cambia el existencial. ✓

Tras la línea 6, $i = j + 1$, e I_0 vale ya que $j < N$ implica $0 \leq j + 1 \leq N$. Por lo tanto, los invariantes son estables.

La demostración de la búsqueda: la t

```
bool solve(vector<int>& A, int v) {  
    bool ans = false;  
    int i = 0;  
    while (i < A.size()) {  
        if (A[i] == v)  
            ans = true;  
        ++i;  
    }  
    return ans;  
}
```

$$I_0: 0 \leq i \leq N$$

$$I_1: \text{ans} = \exists p \in [0..i). A[p] = v$$

Terminación

Se puede garantizar que el ciclo va a terminar ya que i se incrementa de 1 en 1 e I_0 lo acota por N . Finalmente, al salir, la condición $i < N$ es falsa, o sea $i \geq N$; junto con I_0 esto da exactamente $i = N$. Sustituyendo ese valor en I_1 :

$$\text{ans} = \exists p \in [0..N). A[p] = v$$

lo que coincide con la poscondición. Por lo tanto, los invariantes son correctos. ■

La demostración de la búsqueda: el teorema

Teorema 2

La invocación $\text{solve}(A, v)$ para cualquier arreglo A y número v produce true si v está en A y false en caso contrario.

Demostración: es trivial a partir de la correctitud de los invariantes I_0 e I_1 (Teorema 1): la terminación deja $i = N$, y sustituir ese valor en I_1 da $\text{ans} = \exists p \in [0..N). A[p] = v$, que es la poscondición. ■

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios**
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

Ejercicios

1. Cuántas veces aparece un valor

Dado un arreglo $A[0..N)$ de números y un número v , escriba un algoritmo iterativo que cuente cuántas posiciones de A contienen a v , y realice el análisis completo: especificación, invariantes, Teorema 1 con su inicialización, estabilidad y terminación, y Teorema 2.

2. La búsqueda que entrega la posición

Para la variante que devuelve p o -1 : escriba un algoritmo iterativo y proponga sus invariantes I_0 e I_1 . ¿Qué guarda ans mientras no ha encontrado nada?

3. La búsqueda en un arreglo ordenado

La especificación gana la precondition de orden. ¿El algoritmo `solve` la aprovecha en algo? ¿Qué podría hacer un algoritmo que sí la aproveche? (Lo retomaremos con la búsqueda binaria.)

Si se atasca

- En el conteo, la comparación del `if` parte la estabilidad en dos casos: $A[j] = v$ y $A[j] \neq v$. En cada uno hay que decir qué pasa con el conteo del prefijo al agregar la posición j .
- Para la variante con posición, un I_1 posible: $\text{ans} = p$ si ya apareció v en $A[0..i)$ y la primera aparición fue en p ; $\text{ans} = -1$ si v no está en $A[0..i)$.

Para practicar en el navegador

Los mismos pasos, con respuesta inmediata

En las notas del curso hay dos ejercicios interactivos de invariantes, `sumar` y `factorial`, que recorren el camino completo de la clase: la traza por estados, la pareja I_0 e I_1 , y la demostración por inicialización, estabilidad y terminación hasta el Teorema 2. El de `factorial` termina con el experimento de `fact(-1)`.

Dónde

<https://cardel.github.io/notasUniversidad/2026-II/AyG%20PUJ/C2/Ejercicios/>

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes**
- 8 Ejercicios resueltos
- 9 Referencias

Errores comunes al demostrar con invariantes

Los tropiezos de siempre

- **Olvidar I_0 :** sin las cotas del índice, la estabilidad no puede usar que j es un índice válido y la terminación no sabe con qué valor de i termina el ciclo.
- **Proponer la poscondición como invariante:** $\text{ans} = \sum_{j=0}^{N-1} A[j]$ solo vale al final; el invariante debe valer en *todos* los chequeos, empezando por el primero.
- **Probar en la estabilidad lo que se asume:** los invariantes se *asumen* antes de la iteración y se *prueban* después de ella. Confundir esos dos momentos deja la demostración circular. $P(j) \rightarrow P(j+1)$

Errores comunes: y dos más

Y dos más

- **Saltarse la terminación:** inicialización y estabilidad solas no concluyen nada del algoritmo; falta evaluar l_1 en el valor final de i y comparar contra la poscondición.
- **Leer mal la condición del while:** con $i < N$ el ciclo termina en $i = N$; con $i \leq N$, en $i = N + 1$. La cota superior de l_0 sale de ahí.

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos**
- 9 Referencias

Cómo usar esta sección

Tres ciclos con la demostración escrita completa

Intente cada uno antes de pasar a su solución: primero diga qué calcula, después arme la cadena de estados y solo entonces proponga I_0 e I_1 . La demostración viene sola cuando los invariantes están bien.

Qué agrega cada uno

- 1 contarOcurrencias: la estabilidad se parte en casos.
- 2 buscarPosicion: un I_1 que no es una ecuación.
- 3 contarDigitos: un invariante numérico, sin arreglo ni índice.

Los programas de esta sección

Los cuatro programas están en código/

`sumarArreglo.py` —el ejemplo de la clase— junto a `contarOcurrencias.py`, `buscarPosicion.py` y `contarDigitos.py`, con los mismos nombres de estas diapositivas. Cada uno imprime la cadena de estados y contrasta cada fila contra I_1 evaluado a mano, para que el invariante se pueda ver funcionando y no solo leer.

Comprobar un invariante con el computador

Dónde va el assert

Un invariante afirma algo que debe ser cierto en *cada* evaluación de la condición del ciclo. Si eso es cierto, se puede escribir tal cual como un `assert` y ponerlo justo ahí.

```
def sumarArreglo(A):
    i = 0
    ans = 0
    while i < len(A):
        assert 0 <= i <= len(A)           # I0
        assert ans == sumaAMano(A, i)     # I1
        ans = ans + A[i]
        i = i + 1
    assert 0 <= i <= len(A)               # I0 en el ultimo chequeo
    assert ans == sumaAMano(A, i)         # I1 en el ultimo chequeo
    return ans
```

Por qué los assert van dos veces

Los dos grupos de chequeos

Los assert de adentro cubren los chequeos que sí entran al cuerpo. El último chequeo, el que hace *terminar* el ciclo, no entra al cuerpo y por eso necesita su propia copia después del `while`. Ese último es el de la **terminación**: es exactamente el que entrega la poscondición, así que es el que menos se puede dejar por fuera.

El error de poner el assert al final del cuerpo

Ahí también pasa, pero no comprueba lo mismo: estaría verificando el invariante *después* de la iteración y no *antes* de la siguiente condición, y se saltaría el chequeo inicial. El invariante se ancla en la condición del ciclo, no en el cuerpo.

El truco: escribir I_1 dos veces

Una versión lenta pero obviamente correcta

sumaAMano(A, i) no es el algoritmo: es I_1 traducido a código de la forma más directa posible. La gracia está en que sea tan simple que no se pueda dudar de ella. Si el invariante propuesto es correcto, lo que lleva el acumulador y lo que calcula esa versión coinciden en todos los chequeos.

	I_1	Escrito a mano
suma	$\text{ans} = \sum_{j=0}^{i-1} A[j]$	un ciclo que suma $A[0..i)$
ej. 1	$\text{ans} = \{p \in [0..i) : A[p] = v\} $	un ciclo que cuenta
ej. 2	la primera aparición en $A[0..i)$, o -1	un ciclo que recorre el prefijo al revés
ej. 3	$\text{ac} = \lfloor n/10^{\text{ans}} \rfloor$	$n // 10 ** \text{ans}$

Correr no es demostrar

Lo que el computador puede y lo que no

Que ningún assert falle en mil arreglos no demuestra nada: siempre queda el arreglo mil uno. Pero un assert que *falla* refuta el invariante de inmediato, y dice en qué chequeo se rompió. Es la forma más barata de descartar una fórmula falsa antes de gastar media hora demostrándola. La certeza la da la demostración; el computador solo ahorra el tiempo de perseguir un invariante que nunca iba a servir.

Póngalo a fallar a propósito

Cambie I_1 del ejercicio 1 por $\text{ans} = |\{p \in [0..N) : A[p] = v\}|$ — con N en vez de i — y corra otra vez. El assert revienta en el chequeo $i = 0$, que es la inicialización: ese invariante ni siquiera arranca. Verlo fallar en el sitio exacto enseña más que leer que está mal.

Cómo correrlo

```
cd codigo/  
python3 sumarArreglo.py      # imprime la cadena de estados y contrasta I1  
python3 comprobar.py         # los cuatro invariantes, con assert
```

Qué hace comprobar.py

Corre los cuatro algoritmos con los assert puestos sobre *todos* los arreglos de largo 0 a 5 con elementos en $\{0, 1, 2\}$ — son 364, y con las tres opciones de v dan 1 092 casos — y contarDigitos sobre todo n de 1 a 20 000. No usa azar: la prueba es siempre la misma y se puede repetir. Al final deja fallar a propósito el invariante mal escrito para mostrar cómo se ve el mensaje.

Ejercicio 1: cuántas veces aparece un valor

```
0 def contarOcurrencias(A, v):  
1     i = 0  
2     ans = 0  
3     while i < len(A):  
4         if A[i] == v:  
5             ans = ans + 1  
6             i = i + 1  
7     return ans
```

Lo que se pide

Lo mismo de siempre. Preste atención a un detalle nuevo: el cuerpo del ciclo ahora tiene un condicional, y eso cambia cómo se escribe la estabilidad.

Ejercicio 1: especificación e invariantes

```
def contarOcurrencias(A, v):  
    i = 0  
    ans = 0  
    while i < len(A):  
        if A[i] == v:  
            ans = ans + 1  
        i = i + 1  
    return ans
```

Especificación

Entrada: un arreglo $A[0..N)$ de números, $N \geq 0$, y un número v .

Salida: $|\{p \in [0..N) : A[p] = v\}|$.

La cadena de estados (i, ans) con $A = [\underline{4}, 7, 4, 2, \underline{4}]$ y $v = 4$

$(0, 0) \rightarrow (1, 1) \rightarrow (2, 1) \rightarrow (3, 2) \rightarrow (4, 2) \rightarrow (5, 3)$

ans solo se mueve en los chequeos donde el elemento recién visitado era un 4.

Los invariantes

$$I_0 : 0 \leq i \leq N \quad I_1 : \text{ans} = |\{p \in [0..\underline{i}) : A[p] = v\}|$$

Ejercicio 1: Teorema 1 e inicialización

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

Las líneas 1–2 dejan $i = 0$ y $\text{ans} = 0$. Para I_0 : $0 \leq 0 \leq N$. ✓ Para I_1 : el rango $[0., 0)$ es vacío, no contiene ninguna posición y mucho menos una con valor v , así que el conteo correcto es $0 = \text{ans}$. ✓

Ejercicio 1: la estabilidad, por casos

Estabilidad

Iteración arbitraria $i = j$ con $j < N$; se asume $0 \leq j \leq N$ y
 $\text{ans} = |\{p \in [0..j) : A[p] = v\}|$. La comparación de la línea 4 abre dos
 casos: _____

- $A[j] = v$: la línea 5 deja $\text{ans}' = \text{ans} + 1$. Las posiciones de $[0..j+1)$ con valor v son las de $[0..j)$, que por I_1 son ans , más la nueva en $p = j$: en total $\text{ans} + 1$. ✓
- $A[j] \neq v$: la línea 5 no se ejecuta y $\text{ans}' = \text{ans}$. Agregar al rango una posición cuyo valor no es v no cambia el conteo. ✓

En los dos casos l_1 vale para el rango $[0..j+1)$, y la línea 6 deja $i = j + 1$, que es justo el índice de ese rango. Para l_0 : de $j < N$ sale $0 \leq j + 1 \leq N$. Por lo tanto, los invariantes son estables.

Ejercicio 1: la terminación

$$I_0 \quad i < N \wedge p > 0 \leq i \leq N \quad \underline{i = N}$$

```
def contarOcurrencias(A, v):  
    i = 0  
    ans = 0  
    while i < len(A):  
        if A[i] == v:  
            ans = ans + 1  
        i = i + 1  
    return ans
```

Terminación

Se puede garantizar que el ciclo va a terminar ya que i se incrementa de 1 en 1 e I_0 lo acota por N . Finalmente, al salir, la condición $i < N$ es falsa, o sea $i \geq N$; junto con I_0 esto da exactamente $i = N$. Sustituyendo ese valor en I_1 :

$$\text{ans} = |\{ p \in [0..N) : A[p] = v \}|$$

lo que coincide con la poscondición. Por lo tanto, los invariantes son correctos. ■

Ejercicio 1: el teorema del algoritmo

Teorema 2

La invocación `contarOcurrencias(A, v)` produce la cantidad de posiciones de A que contienen a v .

Demostración: es trivial a partir de la correctitud de los invariantes I_0 e I_1 (Teorema 1): la terminación deja $i = N$, y sustituir ese valor en I_1 da la poscondición, que es lo que retorna la línea 7. ■

Ejercicio 1: el tropiezo que hay que evitar

No confunda el invariante con la poscondición

Escribir $I_1 : \text{ans} = |\{p \in [0..N) : A[p] = v\}|$ — con N en vez de i — deja una fórmula que es falsa en todos los chequeos menos el último, así que ni siquiera pasa la inicialización. El invariante habla del *prefijo ya recorrido*; es la terminación, al poner $i = N$, la que lo convierte en la poscondición.

Ejercicio 2: la posición de la primera aparición

```
0 def buscarPosicion(A, v):  
1     i = 0  
2     ans = -1  
3     while i < len(A):  
4         if A[i] == v and ans == -1:  
5             ans = i  
6             i = i + 1  
7     return ans
```

Lo que se pide

Es la variante que quedó planteada como ejercicio: devolver p o -1 .
Antes de proponer I_1 , responda esto: ¿qué guarda `ans` mientras todavía no ha encontrado nada, y por qué la línea 4 pregunta también por `ans`?

Ejercicio 2: especificación e invariantes

Especificación

Entrada: un arreglo $A[0..N)$ de números, $N \geq 0$, y un número v .

Salida: el menor p con $A[p] = v$ si tal p existe en $[0..N)$; -1 si no existe.

La cadena de estados (i, ans) con $A = [3, 8, 5, 8, 1]$ y $v = 8$

$$(0, -1) \rightarrow (1, -1) \rightarrow (2, 1) \rightarrow (3, 1) \rightarrow (4, 1) \rightarrow (5, 1)$$

El segundo 8, en la posición 3, pasa sin dejar rastro: eso es lo que hay que capturar.

Ejercicio 2: un I_1 que no es una ecuación

Los invariantes

$I_0 : 0 \leq i \leq N$. I_1 : si existe $p \in [0..i)$ con $A[p] = v$, entonces ans es el menor de esos p ; si no existe ninguno, $\text{ans} = -1$.

I_1 no es una ecuación sino una disyunción, con un caso por cada situación posible del prefijo. Es legítimo: un invariante es una fórmula lógica, no necesariamente una fórmula aritmética.

Ejercicio 2: Teorema 1 e inicialización

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

Las líneas 1–2 dejan $i = 0$ y $\text{ans} = -1$. Para I_0 : $0 \leq 0 \leq N$. ✓ Para I_1 : el rango $[0., 0)$ es vacío, luego no existe p con $A[p] = v$ y estamos en el segundo caso de la disyunción, que pide exactamente $\text{ans} = -1$. ✓

Ejercicio 2: la estabilidad, lo que se asume

Estabilidad: lo que se asume y lo que se quiere

Se considera una iteración arbitraria $i = j$ (con $j < N$) y se asume que antes de ella valen $0 \leq i \leq N$ y la disyunción I_1 sobre el prefijo $A[0..j)$. El objetivo es mostrar que después de la iteración valen $0 \leq j + 1 \leq N$ y la misma disyunción sobre $A[0..j+1)$.

Ejercicio 2: la estabilidad, los tres casos

Estabilidad: los casos

La línea 4 pregunta por dos cosas a la vez, y de ahí salen tres casos:

- $A[j] = v$ y $\text{ans} = -1$. Por I_1 , v no aparece en $[0..j)$, así que j es la única posición de $[0..j+1)$ con valor v y por lo tanto la menor. La línea 5 deja $\text{ans}' = j$. ✓
- $A[j] = v$ y $\text{ans} \neq -1$. Por I_1 , ans ya es el menor $p \in [0..j)$ con $A[p] = v$, y j es mayor que él; el menor sobre $[0..j+1)$ sigue siendo ans . La condición de la línea 4 es falsa y ans no cambia, que es justo lo que se necesita. ✓
- $A[j] \neq v$. Nada se ejecuta y $\text{ans}' = \text{ans}$; el rango $[0..j+1)$ no gana apariciones de v , así que el caso de la disyunción tampoco cambia. ✓

En los tres casos I_1 vale para $[0..j+1)$. La línea 6 deja $i = j + 1$, e I_0 vale ya que $j < N$ implica $0 \leq j + 1 \leq N$. Por lo tanto, los invariantes son estables.

Ejercicio 2: la terminación y el teorema del algoritmo

Terminación

Se puede garantizar que el ciclo va a terminar ya que i se incrementa de 1 en 1 e l_0 lo acota por N . Finalmente, al salir, la condición $i < N$ es falsa, o sea $i \geq N$; junto con l_0 esto da exactamente $i = N$. Sustituyendo ese valor en l_1 : si v aparece en $A[0..N)$, ans es la posición de su primera aparición, y si no aparece, $\text{ans} = -1$. Es la poscondición. Por lo tanto, los invariantes son correctos. ■

Teorema 2

La invocación `buscarPosicion(A, v)` devuelve la posición de la primera aparición de v en A , o -1 si v no está.

Demostración: es trivial a partir de la correctitud de los invariantes l_0 e l_1 (Teorema 1): la terminación deja $i = N$, y sustituir ese valor en l_1 da la poscondición, que es lo que retorna la línea 7. ■

Ejercicio 2: qué pasa si se quita `ans == -1`

La demostración se rompe en un lugar preciso

Sin esa condición, cada aparición de v sobrescribe `ans` y el algoritmo devuelve la *última*, no la primera. El interesante es dónde se cae la prueba: en el segundo caso de la estabilidad. Allí `ans` pasaría a valer j , pero I_1 exige el *menor* p del prefijo, y ese sigue siendo el viejo. Un invariante bien escrito no solo certifica el algoritmo correcto: señala con el dedo la línea donde falla el incorrecto.

Ejercicio 3: cuántos dígitos tiene un número

```
def contarDigitos(n):  
    ac = n  
    ans = 0  
    while ac > 0:  
        ac = ac // 10  
        ans = ans + 1  
    return ans
```

Lo que se pide

Aquí no hay arreglo ni índice: lo que avanza es una división entera. Antes de leer la solución, construya la cadena de estados (ans , ac) para $n = 4057$ y busque en ella una relación entre ac , n y ans .

Ejercicio 3: especificación e invariantes

Especificación

Entrada: un entero $n \geq 1$. **Salida:** la cantidad de dígitos decimales de n , es decir el único $r \geq 1$ que cumple $10^{r-1} \leq n < 10^r$.

La cadena de estados (ans, ac) con $n = 4057$

$$(0, 4057) \rightarrow (1, 405) \rightarrow (2, 40) \rightarrow (3, 4) \rightarrow (4, 0)$$

Los valores de ac son 4057, 405, 40, 4, 0: en cada fila es n dividido entre una potencia de diez, y el exponente es ans.

Ejercicio 3: el invariante numérico

Los invariantes

$$I_0 : \text{ac} \geq 0$$

$$I_1 : \text{ac} = \left\lfloor \frac{n}{10^{\text{ans}}} \right\rfloor$$

Ejercicio 3: Teorema 1 e inicialización

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

Las líneas 1–2 dejan $ac = n$ y $ans = 0$. Para I_1 :

$\lfloor n/10^0 \rfloor = \lfloor n \rfloor = n = ac$. ✓ Para I_0 : la precondition da $n \geq 1 \geq 0$. ✓

Ejercicio 3: la propiedad que hace falta

La propiedad que hace falta

Para enteros $m \geq 0$ y $a, b \geq 1$ se cumple $\lfloor \lfloor m/a \rfloor / b \rfloor = \lfloor m/(ab) \rfloor$ (CLRS, sección 3.2, p. 54). Dividir dos veces es dividir una vez entre el producto.

Ejercicio 3: la estabilidad

Estabilidad

Se considera una iteración arbitraria en la que valen l_0 e l_1 . Como el cuerpo se ejecuta, la condición de la línea 3 es verdadera: $ac \geq 1$. Las líneas 4–5 producen

$$ac' = \left\lfloor \frac{ac}{10} \right\rfloor = \left\lfloor \frac{\lfloor n/10^{ans} \rfloor}{10} \right\rfloor = \left\lfloor \frac{n}{10^{ans+1}} \right\rfloor, \quad ans' = ans + 1,$$

donde la segunda igualdad es la propiedad de arriba. Es decir $ac' = \lfloor n/10^{ans'} \rfloor$: l_1 vale con los valores nuevos. Para l_0 : el piso de un cociente de no negativos es no negativo, luego $ac' \geq 0$. Por lo tanto, los invariantes son estables. 

Ejercicio 3: la terminación

Terminación

Mientras $ac \geq 1$ se tiene $\lfloor ac/10 \rfloor < ac$, así que ac decrece estrictamente en cada iteración, e l_0 lo mantiene por encima de 0. Un entero que decrece y está acotado por debajo no puede decrecer para siempre: el ciclo termina.

Finalmente, al salir, la condición $ac > 0$ es falsa; con l_0 esto da exactamente $ac = 0$. Y como la precondition pide $n \geq 1$, el primer chequeo sí entró al cuerpo, luego al terminar $ans \geq 1$. ■

Ejercicio 3: el teorema del algoritmo

Teorema 2

Para todo entero $n \geq 1$, contarDigitos(n) produce la cantidad de dígitos decimales de n .

Demostración: se procede de forma directa, apretando ans entre las dos desigualdades que dejan los invariantes. Sea r el valor de ans al salir, con $r \geq 1$ por la terminación. De $\text{ac} = 0$ e I_1 sale $\lfloor n/10^r \rfloor = 0$, es decir $n < 10^r$. Y en el último chequeo que sí entró al cuerpo valía $\text{ans} = r - 1$ con $\text{ac} \geq 1$, o sea $\lfloor n/10^{r-1} \rfloor \geq 1$, es decir $n \geq 10^{r-1}$. Juntando las dos:

$$10^{r-1} \leq n < 10^r,$$

que es exactamente la poscondición. La línea 6 retorna $\text{ans} = r$. ■

Ejercicio 3: fuera de la precondition

¿Y contarDigitos(0)?

Devuelve 0, aunque el 0 se escribe con un dígito. ¿Está mal el algoritmo?
No: la especificación pide $n \geq 1$ y el 0 no es una instancia del problema.
Es el mismo caso de `fact(-1)`: fuera de la precondition no hay promesa.
Si se quisiera aceptar el 0 habría que cambiar primero la especificación, y solo después el código.

Lo que este ejercicio deja

La terminación no fue cosmética: la conclusión salió de usar los dos invariantes a la vez, uno por cada lado de la desigualdad. Ese es el patrón que aparece cada vez que el ciclo cuenta algo que no es un índice.

Contenido

- 1 La especificación de un problema
- 2 Invariantes de ciclo
- 3 La demostración completa: la suma de un arreglo
- 4 Segundo ejemplo: el factorial
- 5 Tercer ejemplo: búsqueda en un arreglo
- 6 Ejercicios
- 7 Errores comunes
- 8 Ejercicios resueltos
- 9 Referencias

```
for (e in lista):  
    ans = ans + e
```

Handwritten red annotations:
- Above the first line: $i=0$
- Above the second line: $ans=0$
- A bracket on the left of the second line, spanning from the first line.
- Below the second line: $i++$

1. Especificacion

Precondicion, Que cumple las entradas

Poscondicion, Que cumple la salida

2. Metodo

1. Estudiar como evoluciona el algoritmo, use una instancia y mire como cambian las variables dentro del ciclo

2. Proponga las invariantes I_0 (indice) I_1 (respuesta) Generalizacion

3. Estudiar inicializacion, estabilidad y terminacion

4. Es importante enunciar los teoremas: Teorema, demostracion y conclusion

Referencias

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, MIT Press, 2009. Sección 2.1, pp. 18–20 (invariantes de ciclo; su *mantenimiento* es la estabilidad del curso).
-  C. Rocha. *Diseño y Análisis de Algoritmos*.